1. Project Title

Evaluating Queue- and Heterogeneity-Aware Aging for CPU Scheduling: A Trace-Driven Ablation Study with Burst-Length Prediction

Working name of the policy: **QH-SJF**. Team Members - Aditya Pratap (24BKT0132) , Deepanshu Kumar Jha (24BKT0152).

2. Problem Statement

When the CPU frees up, the scheduler has to pick the next process. Shortest Job First (SJF) keeps average waiting low, but a long job can be held back by a stream of short ones [1]. Highest Response Ratio Next (HRRN) fixes part of this by raising a job's priority as it waits [6], but it ages every job at a fixed rate, whatever the queue looks like.

QH-SJF changes the aging rate at every dispatch, using two signals from the ready queue: queued work (L) and burst variability (H). It runs the process with the lowest score $B_i / \left(1 + \alpha \, (W_i/B_0)^2\right)$, where W is the time waited and $\alpha = \alpha_{\min} + (\alpha_{\max} - \alpha_{\min}) \, LH$.

The question is whether this beats HRRN and tuned fixed aging, and whether each signal earns its place, including when burst lengths are only predicted. Our research questions:

- Does QH-SJF cut tail waiting (p95, p99, maximum) versus SJF and HRRN, and what does it cost in average waiting?
- Does L help beyond fixed aging, and does H help beyond L?
- Do the results hold with predicted bursts and real traces?
- How sensitive are they to B_0, K, $\alpha_{\min}$, and $\alpha_{\max}$?

3. Motivation

- Averages hide long waits. A few badly delayed jobs matter even when the mean looks good.
- Fixed aging is brittle: too weak under a heavy, mixed backlog and too strong under light load.
- The scheduler already sees the queue at every dispatch, so adapting to it is natural but untested.
- Scheduling proposals are often tested weakly, with weak baselines, tuning on test data, and exact bursts. We want a stricter protocol, and a negative result still counts.

4. Objectives

1. Build a deterministic single-CPU simulator in C11 with one shared engine, so every policy sees identical workloads.
2. Compare QH-SJF with FCFS, SJF, SRTF, HRRN, linear aging, fixed quadratic aging, and L-only and H-only ablations.
3. Use predicted bursts with injected error, not only exact ones [13].
4. Test on synthetic workloads and on workloads derived from public traces.
5. Tune on separate calibration data, freeze the parameters, then test on held-out workloads over many seeds with confidence intervals.
6. Report average and tail metrics honestly, with no novelty or starvation-freedom claim without evidence.

5. Existing System / Existing Approaches

- **FCFS:** simple and never starves a job, but one long job delays everyone behind it.
- **SJF and SRTF:** optimal for average waiting [1] and response time [5], but they can starve long jobs and need known bursts. Predictions help [13].
- **HRRN:** runs the largest $(W+B)/B$, so waiting jobs catch up [6]. The aging rate is fixed. This is our closest rival.
- **Priority aging:** raises priority over time at a fixed rate that must be tuned per workload. Linear and fixed quadratic aging are our baselines.
- **Round Robin and feedback queues:** need no burst knowledge and favour short jobs implicitly [2, 3], but work differently from SJF.
- **Proportional share:** divides the CPU by weight [8]. It targets fairness of share, not short-job efficiency.

As far as we know, none of these adapts aging strength to queued work and burst variability. The survey is not exhaustive, so this still needs the targeted literature search.

6. Literature Survey

This section reviews 15 research publications on CPU scheduling, size-based policies, fairness, prediction, and workload realism. For each one we note what it studies, its main contribution, and why it matters, or falls short, for our purposes. The survey is preliminary and is not a systematic search.

Table 1. Preliminary survey of 15 publications.

No.	Reference	Focus / Method	Main contribution	Relevance / limitation
1	W. E. Smith, "Various Optimizers for Single-Stage Production," *Naval Research Logistics Quarterly*, 1956 [1].	Single-machine job sequencing; shortest processing time rule.	Ordering jobs by shortest processing time minimizes total (and mean) completion time.	Origin of SJF. Assumes all jobs present and exact times known; does not address long-job waiting.
2	F. J. Corbató, M. Merwin-Daggett, R. C. Daley, "An Experimental Time-Sharing System," *Proc. AFIPS Spring Joint Computer Conf.*, 1962 [2].	Compatible Time-Sharing System (CTSS) and its multilevel scheduling.	Long-running processes are moved to lower-priority levels, approximating short-job preference without knowing burst lengths.	Early feedback-queue design. Uses demotion rather than aging; no measure of queue workload.
3	L. Kleinrock, "Analysis of a Time-Shared Processor," *Naval Research Logistics Quarterly*, 1964 [3].	Queueing analysis of a time-shared processor compared with FCFS, with geometric processing times.	Jobs with shorter than average processing requirements spend less time in the queue than under FCFS, and conversely for longer jobs.	Shows time-sharing favours short jobs without knowing sizes. Assumes geometric service times; analytical model.
4	L. Kleinrock, "A Conservation Law for a Wide Class of Queueing Disciplines," *Naval Research Logistics Quarterly*, 1965 [4].	Queueing analysis of a class of service disciplines under Poisson arrivals.	A weighted sum of average waiting times over priority groups is invariant across disciplines in the class, which constrains how waiting can be redistributed.	Explains why trade-offs are unavoidable for disciplines in this class. Analytical; not a scheduler proposal.

Continued on next page

No.	Reference	Focus / Method	Main contribution	Relevance / limitation
5	L. E. Schrage, "A Proof of the Optimality of the Shortest Remaining Processing Time Discipline," *Operations Research*, 1968 [5].	Optimality of SRPT in a single-server queue.	SRPT minimizes mean response time among preemptive disciplines.	Defines the preemptive benchmark (SRTF). Requires known remaining times; says little about tail or fairness.
6	P. Brinch Hansen, "Short Term Scheduling in Multiprogramming Systems," *Proc. 3rd ACM Symposium on Operating System Principles*, 1971 [6].	Scheduling primitives and levels of scheduling in multiprogramming systems.	Defines short-term and medium-term scheduling levels and a set of scheduling primitives.	Historical source for the response-ratio discipline attributed to Brinch Hansen; confirm in the original that the rule appears here.
7	D.-M. Chiu, R. Jain, "Analysis of the Increase and Decrease Algorithms for Congestion Avoidance in Computer Networks," *Computer Networks and ISDN Systems*, 1989 [7].	Comparison of resource-adjustment algorithms using efficiency, fairness, convergence time, and oscillation.	Treats fairness as an explicit, measurable property alongside efficiency when comparing algorithms.	Network setting. A per-process quantity must be defined before a fairness index is used for CPU scheduling; confirm the index definition in the paper.
8	I. Stoica, H. Abdel-Wahab, K. Jeffay, S. K. Baruah, J. E. Gehrke, C. G. Plaxton, "A Proportional Share Resource Allocation Algorithm for Real-Time, Time-Shared Systems," *Proc. IEEE Real-Time Systems Symposium*, 1996 [8].	Proportional-share CPU allocation in discrete time quanta.	Bounds the difference between ideal and actual service received by the size of the time quantum.	Addresses fairness of share and timeliness, not short-job efficiency; no use of queue statistics for aging.
9	M. Harchol-Balter, A. B. Downey, "Exploiting Process Lifetime Distributions for Dynamic Load Balancing," *ACM Trans. Computer Systems*, 1997 [9].	Measurement of Unix process lifetimes.	Reports a heavy-tailed lifetime distribution for longer-running processes, so a few processes account for much of the CPU demand.	Supports burst heterogeneity as a feature of real workloads. Concerns lifetimes, not individual CPU bursts.
10	N. Bansal, M. Harchol-Balter, "Analysis of SRPT Scheduling: Investigating Unfairness," *Proc. ACM SIGMETRICS*, 2001 [10].	Analysis of SRPT versus processor sharing in an M/GI/1 queue.	Shows that under many load and job-size conditions, even large jobs do not do noticeably worse under SRPT.	Cost of favouring short jobs depends on load and size distribution; analytical, single-server, exact sizes.
11	M. Harchol-Balter, B. Schroeder, N. Bansal, M. Agrawal, "Size-Based Scheduling to Improve Web Performance," *ACM Trans. Computer Systems*, 2003 [11].	SRPT-style scheduling for static web requests, implemented and measured.	Large reduction in mean response time with little penalty to large requests.	Request size is known from file size, which is not generally true for CPU bursts; web-server domain.
12	A. Wierman, M. Harchol-Balter, "Classifying Scheduling Policies with Respect to Unfairness in an M/GI/1," *Proc. ACM SIGMETRICS*, 2003 [12].	Formal classification of policies by unfairness relative to processor sharing.	Separates policies into always-fair, sometimes-unfair, and always-unfair categories under load.	Gives a vocabulary for judging QH-SJF's fairness; analytical results need not carry over to non-preemptive simulation.

Continued on next page

No.	Reference	Focus / Method	Main contribution	Relevance / limitation
13	D. Tsafrir, Y. Etsion, D. G. Feitelson, "Backfilling Using System-Generated Predictions Rather than User Runtime Estimates," *IEEE Trans. Parallel and Distributed Systems*, 2007 [13].	Runtime prediction for parallel job scheduling.	System-generated predictions based on past jobs improve backfilling relative to user estimates.	Supports using predicted instead of exact bursts. Parallel-job setting, not single-CPU process scheduling.
14	D. G. Feitelson, D. Tsafrir, D. Krakauer, "Experience with Using the Parallel Workloads Archive," *J. Parallel and Distributed Computing*, 2014 [14].	Use of real job traces in scheduler evaluation.	Describes the archive and pitfalls in trace use, such as anomalies that can distort results.	Guides the trace-driven part of this study; traces are job-level, not CPU-burst-level.
15	S. T. Mirtchev, "Generalised Pollaczek-Khinchine Formula for the Polya/G/1 Queue," *Electronics Letters*, 2017 [15].	Queueing delay formula for Polya/G/1.	Service-time variability enters the mean delay expression.	Theoretical intuition for using burst variability H as a signal; does not propose a scheduler.

6.1 Summary and remaining gap

Taken together, these papers show that favouring short jobs is optimal for average metrics [1, 5], that time-sharing favours short jobs implicitly [3], that conservation results limit how waiting can be shifted between job classes [4], that the penalty to large jobs depends on load and the size distribution [10, 11, 12], that real workloads are heavy-tailed and varied [9], and that predicted runtimes and real traces are needed for a credible evaluation [13, 14]. Feedback queues and proportional-share schedulers tackle long-job delay in other ways [2, 8], and fairness is treated as a measurable property in resource-allocation work [7]. As far as we can tell from these 15 papers, none adapts the aging strength of a non-preemptive SJF score using queued work and burst variability. That is only a working observation, because the survey is not exhaustive. A targeted Google Scholar search for adaptive-aging and SJF-with-aging schedulers is needed before we claim anything about novelty.

7. Research Gap

- Aging strength is fixed in HRRN and in linear or quadratic aging [6].
- Size-based optimality assumes known bursts [1, 5], and prediction has mostly been studied for parallel jobs [13].
- Fairness of size-based policies is mostly analysed with queueing models, not by simulating adaptive aging [10, 12].
- Evaluations are often weak, and public archives describe jobs rather than CPU bursts [14].
- No waiting-time bound is known for this kind of score.

In the sources we reviewed, we found no controlled comparison of queue- and variability-aware aging against HRRN and tuned fixed aging with predicted bursts. This study aims to fill that gap, pending the Google Scholar search.

8. Proposed Solution

At every dispatch, QH-SJF does the following:

1. Take the ready set and compute the queued work Q, the mean burst, and the standard deviation.
2. Compute $L = Q/(Q + KB_0)$ and $H = \sigma/(\mu + \sigma)$.
3. Compute $\alpha = \alpha_{\min} + (\alpha_{\max} - \alpha_{\min})\,LH$.
4. Score each process: $B_i/\left(1 + \alpha\,(W_i/B_0)^2\right)$, with $W_i = t - A_i$.
5. Run the lowest score to completion. Ties go to the earlier arrival, then the lower process ID.

α always stays between $\alpha_{\min}$ and $\alpha_{\max}$. With one job or equal bursts, H is zero and aging stays weak. With a heavy, varied queue, aging grows. Starting values are $\alpha_{\min} = 0.10$, $\alpha_{\max} = 1.00$, $K = 5$, and B_0 equal to the median burst in a calibration set, all tuned on calibration data only. Policies see predicted bursts, and true bursts are used only to run jobs.

Our hypothesis, not a result, is that QH-SJF may cut tail waiting on heterogeneous, loaded workloads for a small cost in average waiting. We will try to prove a waiting bound under stated assumptions, and otherwise report only measured tails.

9. Proposed Architecture / Block Diagram

Workloads are validated, and the experiment controller runs every policy through one shared engine. Results flow into metrics, CSV files, and statistics.

10. Dataset / Source of Data

- **Hand-checkable cases:** tiny workloads (a single job, an idle gap, equal arrivals and so on) to verify the simulator.
- **Synthetic workloads:** a seeded generator with seven families (homogeneous, heterogeneous, long-job stress, heavy backlog, bursty arrivals, light load, mixed stress). Parameters are fixed before test runs. We propose 100 processes per run and 30 seeds.
- **Public traces:** the Parallel Workloads Archive [14]. Submit time becomes arrival and run time becomes burst. We drop the recorded wait time, jobs with run time of zero or less, and the processor count, cut each log into windows, use at least two logs, and follow each log's terms of use.
- **Caveat:** those logs describe parallel jobs, not CPU bursts, so the conversion is an approximation.
- **Format:** CSV with `pid`, `arrival`, and `burst`. Calibration and held-out sets use separate seeds.

11. Evaluation Parameters

- **Metrics:** average, p95, p99, and maximum waiting (nearest-rank percentiles), turnaround, slowdown (turnaround divided by burst), throughput, and CPU utilization. Response time equals waiting time in this model.
- **Parameters:** B_0, K, $\alpha_{\min}$, $\alpha_{\max}$, β (linear aging), and α_{fixed}, with sensitivity tested at low, middle, and high values.
- **Factors:** workload family, load level, prediction error, and policy.
- **Statistics:** mean, median, and 95 percent confidence intervals across seeds, with paired Wilcoxon tests on identical seeds.

12. Expected Outcomes

- **Deliverables:** a tested C11 simulator, raw CSV results, tables and plots, and an honest written analysis.

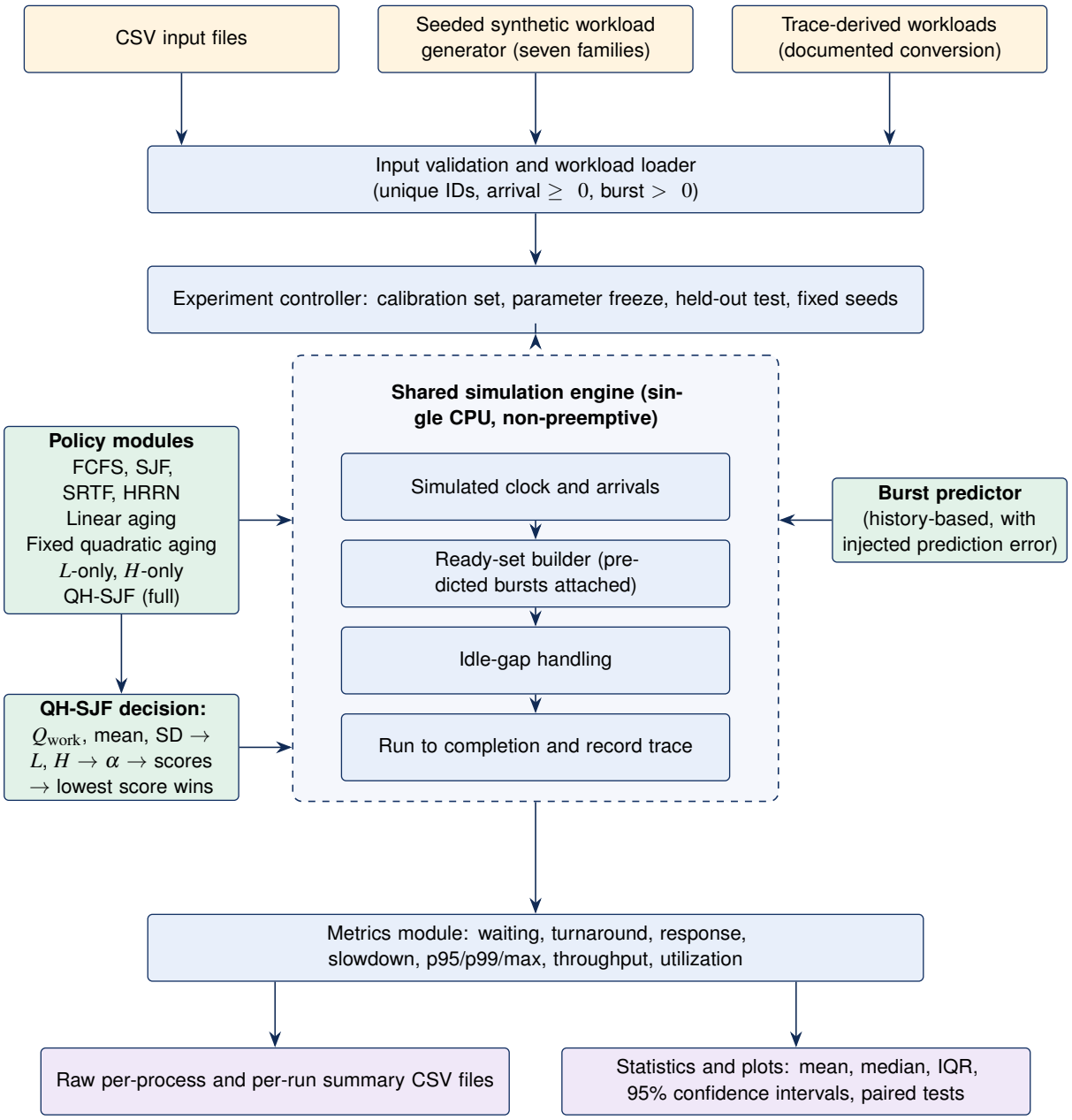


Figure 1. Block diagram of the proposed simulator and experiment pipeline.

- If QH-SJF cuts tail waiting for a small average cost, adaptation may help on those loads.
- If it matches tuned fixed aging or L-only, the extra signals are not worth the complexity.
- If HRRN wins, that is still a valid result.
- If gains vanish under prediction error, the policy needs accurate bursts.
- **Limits:** it is a simulation, not a kernel, with one CPU and single-burst jobs, and the trace conversion is approximate.

2pt

References

[1] W. E. Smith, "Various optimizers for single-stage production," *Naval Research Logistics Quarterly*, vol. 3, no. 1–2, pp. 59–66, 1956.

[2] F. J. Corbató, M. Merwin-Daggett, and R. C. Daley, "An experimental time-sharing system," in *Proceedings of the AFIPS Spring Joint Computer Conference (SJCC '62)*, vol. 21, pp. 335–344, 1962.

[3] L. Kleinrock, "Analysis of a time-shared processor," *Naval Research Logistics Quarterly*, vol. 11, no. 1, pp. 59–73, 1964.

[4] L. Kleinrock, "A conservation law for a wide class of queueing disciplines," *Naval Research Logistics Quarterly*, vol. 12, no. 2, pp. 181–192, 1965.

[5] L. E. Schrage, "A proof of the optimality of the shortest remaining processing time discipline," *Operations Research*, vol. 16, no. 3, pp. 687–690, 1968.

[6] P. Brinch Hansen, "Short term scheduling in multiprogramming systems," in *Proceedings of the 3rd ACM Symposium on Operating Systems Principles (SOSP '71)*, pp. 1–10, 1971.

[7] D.-M. Chiu and R. Jain, "Analysis of the increase and decrease algorithms for congestion avoidance in computer networks," *Computer Networks and ISDN Systems*, vol. 17, no. 1, pp. 1–14, 1989.

[8] I. Stoica, H. Abdel-Wahab, K. Jeffay, S. K. Baruah, J. E. Gehrke, and C. G. Plaxton, "A proportional share resource allocation algorithm for real-time, time-shared systems," in *Proceedings of the 17th IEEE Real-Time Systems Symposium (RTSS '96)*, pp. 288–299, 1996.

[9] M. Harchol-Balter and A. B. Downey, "Exploiting process lifetime distributions for dynamic load balancing," *ACM Transactions on Computer Systems (TOCS)*, vol. 15, no. 3, pp. 253–285, 1997.

[10] N. Bansal and M. Harchol-Balter, "Analysis of SRPT scheduling: Investigating unfairness," in *Proceedings of the ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems (SIGMETRICS '01)*, pp. 224–235, 2001.

[11] M. Harchol-Balter, B. Schroeder, N. Bansal, and M. Agrawal, "Size-based scheduling to improve web performance," *ACM Transactions on Computer Systems (TOCS)*, vol. 21, no. 2, pp. 207–233, 2003.

[12] A. Wierman and M. Harchol-Balter, "Classifying scheduling policies with respect to unfairness in an M/GI/1," in *Proceedings of the ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems (SIGMETRICS '03)*, pp. 200–211, 2003.

[13] D. Tsafrir, Y. Etsion, and D. G. Feitelson, "Backfilling using system-generated predictions rather than user runtime estimates," *IEEE Transactions on Parallel and Distributed Systems*, vol. 18, no. 6, pp. 789–803, 2007.

[14] D. G. Feitelson, D. Tsafrir, and D. Krakauer, "Experience with using the Parallel Workloads Archive," *Journal of Parallel and Distributed Computing*, vol. 74, no. 10, pp. 2967–2986, 2014.

[15] S. T. Mirtchev, "Generalised Pollaczek-Khinchine formula for the Polya/G/1 queue," *Electronics Letters*, vol. 53, no. 20, pp. 1362–1364, 2017.